Lab: Refactor Azure Databricks VNet Injection into Reusable Terraform Modules with Remote State and Dev/Prod Environments

Description

In this lab, you will take the VNet injection lab (a workspace in a custom VNet with two subnets, an NSG and a NAT Gateway) and turn it into production-style Terraform code.

You will make three changes:

- Reusable modules → the code is split into three modules: network, workspace and unity_catalog. Each module does one job and can be reused by any team.
- Remote state → Terraform state is stored in an Azure Storage account, as in the remote state lab. The state is shared, locked during changes and never lost with a laptop.
- Multiple environments → dev and prod are deployed from the same code. Only the tfvars file and the state file differ.

This is how platform teams keep environments consistent. A change is written once, tested in dev, then promoted to prod with confidence that the code is identical.

Learning Objectives

By the end of this lab, you will be able to:

1. Store Terraform state in Azure Storage using Microsoft Entra ID authentication instead of access keys.
2. Use partial backend configuration so each environment has its own state file.
3. Design modules with clear input variables and outputs.
4. Declare required_providers inside a module that uses a non-HashiCorp provider.
5. Compose the modules in a root module with shared naming and tagging conventions.
6. Deploy dev and prod from the same code using separate tfvars files.
7. Promote a change from dev to prod and confirm that each environment is isolated.

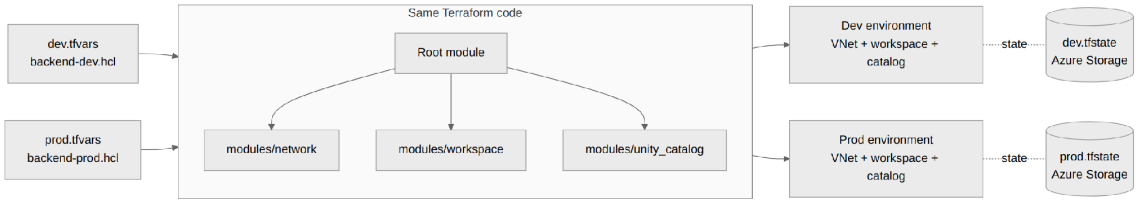
Prerequisites

Make sure you have the following before starting:

- A valid **Azure subscription** where you have **Owner**, or **Contributor** plus **User Access Administrator**.
- **Terraform 1.9 or later** and **Azure CLI** installed locally.
- A **bash** shell. On Windows, use **WSL** or **Git Bash**.
- Completion of the earlier labs: *Store Terraform State in Remote Storage*, *Deploy Azure Databricks with VNet Injection* and *Set Up Unity Catalog on Azure Databricks*.

This lab assumes your region already has a Unity Catalog metastore, which is true for most accounts. Both workspaces are attached to it automatically.

Architecture



Project Structure

By the end of the lab, your project will look like this:

```
databricks_modular/
├── backend.tf
├── providers.tf
├── variables.tf
├── main.tf
├── outputs.tf
├── deploy.sh
├── environments/
│   ├── backend-dev.hcl
│   ├── backend-prod.hcl
│   ├── dev.tfvars
│   └── prod.tfvars
```

```
└── modules/
    ├── network/          main.tf, variables.tf, outputs.tf
    ├── workspace/        main.tf, variables.tf, outputs.tf
    └── unity_catalog/    versions.tf, main.tf, variables.tf, outputs.tf
```

How Dev and Prod Differ

Setting	Dev	Prod
Resource group	rg-dbxlab-dev	rg-dbxlab-prod
VNet address space	10.10.0.0/16	10.20.0.0/16
Host and container subnets	/24 each, up to about 250 nodes	/23 each, up to about 500 nodes
Catalog	dev_catalog	prod_catalog
Schemas	bronze, silver, gold, sandbox	bronze, silver, gold
State file	databricks/dev.tfstate	databricks/prod.tfstate

Everything else, including the code, the modules and the security settings, is identical.

Step by Step Guide

Part A: Set Up Remote State

Step 1: Create the State Storage Account

The state storage must exist **before** Terraform can use it, so you create it once with the Azure CLI. This is the only part of the lab not managed by Terraform.

```
az login
az account set --subscription "<your-subscription-id>"

STATE_RG="rg-tfstate-lab"
STATE_SA="sttfstate$(openssl rand -hex 3)"

az group create --name $STATE_RG --location "West Europe"
```

```
az storage account create \
  --name $STATE_SA \
  --resource-group $STATE_RG \
  --location "West Europe" \
  --sku Standard_LRS \
  --min-tls-version TLS1_2 \
  --allow-blob-public-access false

az storage account blob-service-properties update \
  --account-name $STATE_SA \
  --resource-group $STATE_RG \
  --enable-versioning true

az role assignment create \
  --role "Storage Blob Data Contributor" \
  --assignee $(az ad signed-in-user show --query id -o tsv) \
  --scope $(az storage account show -n $STATE_SA -g $STATE_RG --query id -o tsv)

echo "State storage account: $STATE_SA"
```

Wait about a minute for the role to take effect, then create the container:

```
az storage container create \
  --name tfstate \
  --account-name $STATE_SA \
  --auth-mode login
```

Explanation:

- Blob versioning → every state change keeps the previous version, so a corrupted or deleted state file can be restored.
- Storage Blob Data Contributor → lets Terraform read and write state with your Microsoft Entra ID identity. No storage account keys are needed.
- --auth-mode login → uses your identity instead of an account key, which confirms the role assignment works.

Note the storage account name printed at the end. You need it in Step 2.

Step 2: Create the Project and Backend Configuration

Create the project folders:

```
mkdir -p databricks_modular/environments
mkdir -p databricks_modular/modules/{network,workspace,unity_catalog}
cd databricks_modular
```

Create a file named backend.tf:

```
terraform {
  backend "azurerm" {}
}
```

Create environments/backend-dev.hcl. Replace the storage account name with yours:

```
resource_group_name  = "rg-tfstate-lab"
storage_account_name = "sttfstatexxxxxx"
container_name       = "tfstate"
key                  = "databricks/dev.tfstate"
use_azuread_auth     = true
```

Create environments/backend-prod.hcl:

```
resource_group_name  = "rg-tfstate-lab"
storage_account_name = "sttfstatexxxxxx"
container_name       = "tfstate"
key                  = "databricks/prod.tfstate"
use_azuread_auth     = true
```

Explanation:

- Empty backend block → this is a partial configuration. The details are supplied at terraform init time, so the same code can point to different state files.
- key → the only line that differs. Dev and prod each get their own state file, so a change to one can never touch the other.
- use_azuread_auth = true → Terraform authenticates to the state storage with your identity instead of an access key.

Part B: Build the Modules

Each module has three files: variables.tf (its inputs), main.tf (its resources) and outputs.tf (the values it hands back). Modules never contain provider blocks or hard-coded names. Everything that changes between environments comes in through variables.

Step 3: Create the Network Module

This module contains the VNet, the two delegated subnets, the NSG and the NAT Gateway from the VNet injection lab.

Create modules/network/variables.tf:

```
variable "name_prefix" {
  type        = string
  description = "Prefix for resource names, e.g. dbxlab-dev"
}

variable "location" {
  type = string
}

variable "resource_group_name" {
  type = string
}

variable "vnet_address_space" {
  type = list(string)

  validation {
    condition     = alltrue([for cidr in var.vnet_address_space : can(cidrnetmask(cidr))])
    error_message = "Every entry in vnet_address_space must be a valid IPv4 CIDR block."
  }
}

variable "host_subnet_prefix" {
  type = list(string)
}
```

```
variable "container_subnet_prefix" {
  type = list(string)
}

variable "nat_idle_timeout_minutes" {
  type    = number
  default = 10
}

variable "tags" {
  type    = map(string)
  default = {}
}
```

Create modules/network/main.tf:

```
locals {
  subnets = {
    host      = var.host_subnet_prefix
    container = var.container_subnet_prefix
  }
}

resource "azurerm_virtual_network" "this" {
  name                = "vnet-${var.name_prefix}"
  address_space       = var.vnet_address_space
  location            = var.location
  resource_group_name = var.resource_group_name
  tags                = var.tags
}

resource "azurerm_subnet" "databricks" {
  for_each = local.subnets

  name                 = "snet-${var.name_prefix}-${each.key}"
  resource_group_name  = var.resource_group_name
```

```
  virtual_network_name = azurerm_virtual_network.this.name
  address_prefixes     = each.value

  delegation {
    name = "databricks-${each.key}-delegation"

    service_delegation {
      name = "Microsoft.Databricks/workspaces"
      actions = [
        "Microsoft.Network/virtualNetworks/subnets/join/action",
        "Microsoft.Network/virtualNetworks/subnets/prepareNetworkPolicies/action",
        "Microsoft.Network/virtualNetworks/subnets/unprepareNetworkPolicies/action",
      ]
    }
  }
}

resource "azurerm_network_security_group" "this" {
  name                = "nsg-${var.name_prefix}"
  location            = var.location
  resource_group_name = var.resource_group_name
  tags                = var.tags
}

resource "azurerm_subnet_network_security_group_association" "databricks" {
  for_each = azurerm_subnet.databricks

  subnet_id                 = each.value.id
  network_security_group_id = azurerm_network_security_group.this.id
}

resource "azurerm_public_ip" "nat" {
  name                = "pip-nat-${var.name_prefix}"
  location            = var.location
  resource_group_name = var.resource_group_name
  allocation_method   = "Static"
```

```
  sku                 = "Standard"
  tags                = var.tags
}

resource "azurerm_nat_gateway" "this" {
  name                    = "nat-${var.name_prefix}"
  location                = var.location
  resource_group_name     = var.resource_group_name
  sku_name                = "Standard"
  idle_timeout_in_minutes = var.nat_idle_timeout_minutes
  tags                    = var.tags
}

resource "azurerm_nat_gateway_public_ip_association" "this" {
  nat_gateway_id       = azurerm_nat_gateway.this.id
  public_ip_address_id = azurerm_public_ip.nat.id
}

resource "azurerm_subnet_nat_gateway_association" "databricks" {
  for_each = azurerm_subnet.databricks

  subnet_id      = each.value.id
  nat_gateway_id = azurerm_nat_gateway.this.id
}
```

Create modules/network/outputs.tf:

```
output "vnet_id" {
  value = azurerm_virtual_network.this.id
}

output "host_subnet_name" {
  value = azurerm_subnet.databricks["host"].name
}

output "container_subnet_name" {
  value = azurerm_subnet.databricks["container"].name
```

```
}

output "host_nsg_association_id" {
  value = azurerm_subnet_network_security_group_association.databricks["host"].id
}

output "container_nsg_association_id" {
  value = azurerm_subnet_network_security_group_association.databricks["container"].id
}

output "nat_public_ip" {
  value = azurerm_public_ip.nat.ip_address
}
```

Explanation:

- name_prefix → every name is built from it, so the same module creates vnet-dbxlab-dev in dev and vnet-dbxlab-prod in prod.
- validation → the module rejects a bad CIDR at plan time with a clear message, before anything is deployed.
- Outputs → the workspace module needs the VNet ID, subnet names and NSG association IDs. Outputs are the only way to pass them out of the module.

Step 4: Create the Workspace Module

Create modules/workspace/variables.tf:

```
variable "name" {
  type = string
}

variable "location" {
  type = string
}

variable "resource_group_name" {
  type = string
}
```

```
variable "managed_resource_group_name" {
  type = string
}

variable "sku" {
  type    = string
  default = "premium"

  validation {
    condition     = contains(["standard", "premium", "trial"], var.sku)
    error_message = "sku must be standard, premium or trial."
  }
}

variable "virtual_network_id" {
  type = string
}

variable "host_subnet_name" {
  type = string
}

variable "container_subnet_name" {
  type = string
}

variable "host_nsg_association_id" {
  type = string
}

variable "container_nsg_association_id" {
  type = string
}

variable "tags" {
```

```
  type    = map(string)
  default = {}
}
```

Create modules/workspace/main.tf:

```
resource "azurerm_databricks_workspace" "this" {
  name                        = var.name
  resource_group_name         = var.resource_group_name
  location                    = var.location
  sku                         = var.sku
  managed_resource_group_name = var.managed_resource_group_name

  custom_parameters {
    no_public_ip        = true
    virtual_network_id  = var.virtual_network_id
    public_subnet_name  = var.host_subnet_name
    private_subnet_name = var.container_subnet_name

    public_subnet_network_security_group_association_id  = var.host_nsg_association_id
    private_subnet_network_security_group_association_id = var.container_nsg_association_id
  }

  tags = var.tags
}
```

Create modules/workspace/outputs.tf:

```
output "id" {
  value = azurerm_databricks_workspace.this.id
}

output "workspace_url" {
  value = azurerm_databricks_workspace.this.workspace_url
}

output "workspace_id" {
  value = azurerm_databricks_workspace.this.workspace_id
}
```

Explanation:

- Loose coupling → the workspace module does not know how the network was built. It only needs IDs and names, so it could also be used with a network built by another team.
- no_public_ip = true → secure cluster connectivity is fixed inside the module, so no environment can switch it off by accident.

Step 5: Create the Unity Catalog Module

This module packages the Unity Catalog lab: ADLS Gen2 storage, the Access Connector, a storage credential, external locations, a catalog, its schemas and a catalog grant.

Create modules/unity_catalog/versions.tf:

```
terraform {
  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
    }

    random = {
      source = "hashicorp/random"
    }

    time = {
      source = "hashicorp/time"
    }

    databricks = {
      source = "databricks/databricks"
    }
  }
}
```

Why this file matters: without it, Terraform assumes every provider comes from the hashicorp namespace and looks for hashicorp/databricks, which does not exist. Any module that uses a non-HashiCorp provider must declare its source. Versions are pinned once, in the root module.

Create modules/unity_catalog/variables.tf:

```
variable "name_prefix" {
  type = string
}

variable "location" {
  type = string
}

variable "resource_group_name" {
  type = string
}

variable "catalog_name" {
  type = string
}

variable "schemas" {
  type        = map(string)
  description = "Schema name => comment"
}

variable "catalog_user_group" {
  type    = string
  default = "account users"
}

variable "tags" {
  type    = map(string)
  default = {}
}
```

Create modules/unity_catalog/main.tf:

```
# ---------- Azure side ----------
resource "random_string" "suffix" {
  length  = 6
```

```
  special = false
  upper   = false
}

resource "azurerm_storage_account" "this" {
  name                     = "st${replace(var.name_prefix, "-", "")}${random_string.suffix.result}"
  resource_group_name      = var.resource_group_name
  location                 = var.location
  account_tier             = "Standard"
  account_replication_type = "LRS"
  account_kind             = "StorageV2"
  is_hns_enabled           = true
  min_tls_version          = "TLS1_2"
  tags                     = var.tags
}

resource "azurerm_storage_container" "this" {
  for_each = toset(["managed", "landing"])

  name                  = each.key
  storage_account_id    = azurerm_storage_account.this.id
  container_access_type = "private"
}

resource "azurerm_databricks_access_connector" "this" {
  name                = "ac-${var.name_prefix}"
  resource_group_name = var.resource_group_name
  location            = var.location

  identity {
    type = "SystemAssigned"
  }

  tags = var.tags
}
```

```
resource "azurerm_role_assignment" "connector_storage" {
  scope                = azurerm_storage_account.this.id
  role_definition_name = "Storage Blob Data Contributor"
  principal_id         = azurerm_databricks_access_connector.this.identity[0].principal_id
}

resource "time_sleep" "wait_for_rbac" {
  create_duration = "60s"
  depends_on      = [azurerm_role_assignment.connector_storage]
}

# ---------- Unity Catalog side ----------
resource "databricks_storage_credential" "this" {
  name = "cred-${var.name_prefix}"

  azure_managed_identity {
    access_connector_id = azurerm_databricks_access_connector.this.id
  }

  force_destroy = true
  depends_on    = [time_sleep.wait_for_rbac]
}

resource "databricks_external_location" "this" {
  for_each = azurerm_storage_container.this

  name            = "loc-${var.name_prefix}-${each.key}"
  url             =
"abfss://${each.value.name}@${azurerm_storage_account.this.name}.dfs.core.windows.net/"
  credential_name = databricks_storage_credential.this.name
  force_destroy   = true
}

resource "databricks_catalog" "this" {
  name         = var.catalog_name
  storage_root  = "${databricks_external_location.this["managed"].url}${var.catalog_name}"
```

```
  comment      = "Managed by Terraform (${var.name_prefix})"
  force_destroy = true
}

resource "databricks_schema" "this" {
  for_each = var.schemas

  catalog_name  = databricks_catalog.this.name
  name          = each.key
  comment       = each.value
  force_destroy = true
}

resource "databricks_grants" "catalog" {
  catalog = databricks_catalog.this.name

  grant {
    principal  = var.catalog_user_group
    privileges = ["USE_CATALOG", "USE_SCHEMA", "SELECT"]
  }
}
```

Create modules/unity_catalog/outputs.tf:

```
output "storage_account_name" {
  value = azurerm_storage_account.this.name
}

output "catalog_name" {
  value = databricks_catalog.this.name
}

output "schema_names" {
  value = [for schema in databricks_schema.this : schema.name]
}
```

Explanation:

- Metastore-wide names → dev and prod share the regional metastore, so credential, location and catalog names must be unique across both. The name_prefix guarantees this.
- time_sleep → the Unity Catalog lab relied on a pause between two separate applies. Inside one module, time_sleep provides that pause instead.
- schemas as a map → each environment passes its own list of schemas. Adding a schema is a one-line change in a tfvars file.

Part C: Compose the Root Module

Step 6: Configure the Providers

Create providers.tf in the project root:

```
terraform {
 required_version = ">= 1.9"

 required_providers {
  azurerm = {
   source  = "hashicorp/azurerm"
   version = "5.6.0"
  }

  random = {
   source  = "hashicorp/random"
   version = "~> 3.7"
  }

  time = {
   source  = "hashicorp/time"
   version = "~> 0.13"
  }

  databricks = {
   source  = "databricks/databricks"
   version = "1.132.0"
  }
```

```
  }
}

provider "azurerm" {
  features {}
  subscription_id = var.subscription_id

  resource_providers_to_register = [
    "Microsoft.Databricks",
    "Microsoft.Network",
    "Microsoft.Storage",
  ]
}

provider "databricks" {
  host                        = module.workspace.workspace_url
  azure_workspace_resource_id = module.workspace.id
}
```

Explanation:

- Versions live in the root → modules declare only the provider source. The root pins exact versions, so every module in an environment uses the same provider version.
- subscription_id from a variable → dev and prod can live in different subscriptions, which is common in enterprises.
- Databricks provider from module outputs → the provider connects to whichever workspace this environment creates.

Step 7: Create the Root Variables, Main and Outputs

Create variables.tf:

```
variable "subscription_id" {
  type = string
}

variable "project" {
  type    = string
```

```
  default = "dbxlab"

  validation {
    condition     = can(regex("^[a-z0-9]{3,8}$", var.project))
    error_message = "project must be 3-8 lowercase letters or digits (it is used in storage account
names)."
  }
}

variable "environment" {
  type = string

  validation {
    condition     = contains(["dev", "prod"], var.environment)
    error_message = "environment must be dev or prod."
  }
}

variable "location" {
  type    = string
  default = "West Europe"
}

variable "vnet_address_space" {
  type = list(string)
}

variable "host_subnet_prefix" {
  type = list(string)
}

variable "container_subnet_prefix" {
  type = list(string)
}

variable "catalog_schemas" {
```

```
  type = map(string)
}

variable "tags" {
  type    = map(string)
  default = {}
}
```

Create main.tf:

```
locals {
  name_prefix = "${var.project}-${var.environment}"

  tags = merge(var.tags, {
    project     = var.project
    environment = var.environment
    managed_by  = "terraform"
  })
}

resource "azurerm_resource_group" "this" {
  name     = "rg-${local.name_prefix}"
  location = var.location
  tags     = local.tags
}

module "network" {
  source = "./modules/network"

  name_prefix             = local.name_prefix
  location                = azurerm_resource_group.this.location
  resource_group_name     = azurerm_resource_group.this.name
  vnet_address_space      = var.vnet_address_space
  host_subnet_prefix      = var.host_subnet_prefix
  container_subnet_prefix = var.container_subnet_prefix
  tags                    = local.tags
}
```

```
module "workspace" {
  source = "./modules/workspace"

  name                        = "dbw-${local.name_prefix}"
  location                    = azurerm_resource_group.this.location
  resource_group_name         = azurerm_resource_group.this.name
  managed_resource_group_name  = "rg-${local.name_prefix}-managed"
  virtual_network_id          = module.network.vnet_id
  host_subnet_name            = module.network.host_subnet_name
  container_subnet_name       = module.network.container_subnet_name
  host_nsg_association_id     = module.network.host_nsg_association_id
  container_nsg_association_id = module.network.container_nsg_association_id
  tags                        = local.tags

  depends_on = [module.network]
}

module "unity_catalog" {
  source = "./modules/unity_catalog"

  name_prefix        = local.name_prefix
  location           = azurerm_resource_group.this.location
  resource_group_name = azurerm_resource_group.this.name
  catalog_name       = "${var.environment}_catalog"
  schemas            = var.catalog_schemas
  tags               = local.tags

  depends_on = [module.workspace]
}
```

Create outputs.tf:

```
output "environment" {
  value = var.environment
}
```

```
output "databricks_workspace_url" {
  value = "https://${module.workspace.workspace_url}"
}

output "nat_public_ip" {
  value = module.network.nat_public_ip
}

output "uc_storage_account_name" {
  value = module.unity_catalog.storage_account_name
}

output "catalog_name" {
  value = module.unity_catalog.catalog_name
}

output "schema_names" {
  value = module.unity_catalog.schema_names
}
```

Explanation:

- One naming rule → name_prefix produces names like rg-dbxlab-dev and dbw-dbxlab-prod everywhere.
- merge() → every resource gets the environment's own tags plus three standard tags, which makes cost reports and audits easy.
- Module outputs as inputs → module.network.vnet_id flows into the workspace module. Terraform works out the order from these references.
- depends_on → makes sure the NAT Gateway is attached before the workspace is created, and that Unity Catalog waits for the workspace.

Step 8: Create the Environment Files

Create environments/dev.tfvars:

```
subscription_id = "<your-subscription-id>"
project         = "dbxlab"
environment     = "dev"
```

```
location        = "West Europe"

vnet_address_space      = ["10.10.0.0/16"]
host_subnet_prefix      = ["10.10.1.0/24"]
container_subnet_prefix = ["10.10.2.0/24"]

catalog_schemas = {
  bronze  = "Raw data as ingested"
  silver  = "Cleaned and conformed data"
  gold    = "Business-level aggregates"
  sandbox = "Scratch space for experiments"
}

tags = {
  cost_center = "data-platform-dev"
  owner       = "data-platform-team"
}
```

Create environments/prod.tfvars:

```
subscription_id = "<your-subscription-id>"
project         = "dbxlab"
environment     = "prod"
location        = "West Europe"

vnet_address_space      = ["10.20.0.0/16"]
host_subnet_prefix      = ["10.20.0.0/23"]
container_subnet_prefix = ["10.20.2.0/23"]

catalog_schemas = {
  bronze = "Raw data as ingested"
  silver = "Cleaned and conformed data"
  gold   = "Business-level aggregates"
}

tags = {
  cost_center = "data-platform-prod"
```

```
  owner       = "data-platform-team"
  criticality = "high"
}
```

Dev and prod use non-overlapping address spaces, so they can be peered to a shared hub later without conflicts.

Step 9: Create the Deploy Script

The most dangerous mistake with this setup is running prod.tfvars against the dev state. Terraform would then try to rename or replace dev resources to match prod. A small script removes that risk by always pairing the backend file and the tfvars file.

Create deploy.sh:

```
#!/usr/bin/env bash
set -euo pipefail

USAGE="Usage: ./deploy.sh <dev|prod> <plan|apply|destroy|output> [extra terraform args]"
ENV="${1:?$USAGE}"
ACTION="${2:?$USAGE}"
shift 2

if [[ "$ENV" != "dev" && "$ENV" != "prod" ]]; then
  echo "Environment must be dev or prod" >&2
  exit 1
fi

if [[ "$ENV" == "prod" && ( "$ACTION" == "apply" || "$ACTION" == "destroy" ) ]]; then
  read -r -p "You are about to run '$ACTION' on PROD. Type 'prod' to continue: " CONFIRM
  [[ "$CONFIRM" == "prod" ]] || { echo "Cancelled."; exit 1; }
fi

terraform init -input=false -reconfigure -backend-config="environments/backend-${ENV}.hcl" > /dev/null
echo "==> ${ACTION} on ${ENV} (state: databricks/${ENV}.tfstate)"

if [[ "$ACTION" == "output" ]]; then
  terraform output "$@"
```

```
else
  terraform "$ACTION" -var-file="environments/${ENV}.tfvars" "$@"
fi
```

Make it executable:

```
chmod +x deploy.sh
```

Explanation:

- -reconfigure → switches Terraform to the chosen environment's state file every time.
- Paired files → backend-dev.hcl always goes with dev.tfvars, and backend-prod.hcl with prod.tfvars.
- Prod confirmation → any apply or destroy on prod asks you to type prod first.
- Extra arguments → anything after the action is passed to Terraform, for example -target or --auto-approve.

Part D: Deploy and Operate the Environments

Step 10: Deploy Dev

If several participants share one Databricks account, set project in both tfvars files to something unique, such as dbx plus your initials. Unity Catalog names must be unique across the shared metastore.

Format the code, then run the first deployment in two stages:

```
terraform fmt -recursive


./deploy.sh dev apply -target=module.workspace
./deploy.sh dev apply
```

Explanation:

- First stage (-target) → creates the resource group, network and workspace. The Databricks provider cannot connect until the workspace exists, so this bootstrap step is needed once per environment. Terraform prints a warning about -target; that is expected here.
- Second stage → creates everything else, including the Unity Catalog module. From now on, a single ./deploy.sh dev apply is enough.

View the outputs:

```
./deploy.sh dev output
```

The first dev deployment takes about 15 minutes.

Step 11: Deploy Prod from the Same Code

```
./deploy.sh prod apply -target=module.workspace
./deploy.sh prod apply
./deploy.sh prod output
```

Type prod when the script asks for confirmation. Nothing in the code changed. Only the backend file and tfvars file are different.

Step 12: Verify Isolation and Promote a Change

12.1 Check the state files

```
az storage blob list \
  --account-name <your-state-storage-account> \
  --container-name tfstate \
  --auth-mode login \
  --query "[].name" --output table
```

You see databricks/dev.tfstate and databricks/prod.tfstate: one state file per environment.

12.2 Check the resources

```
az group list \
  --query "[?tags.managed_by=='terraform'].{name:name, environment:tags.environment}" \
  --output table
```

You see rg-dbxlab-dev and rg-dbxlab-prod, each tagged with its environment.

12.3 Look inside a state file

```
./deploy.sh dev output
terraform state list
```

Resources now have module addresses, for example module.network.azurerm_subnet.databricks["host"] and module.unity_catalog.databricks_catalog.this.

12.4 Check the catalogs

Open the dev workspace and click **Catalog**. You see **dev_catalog** with four schemas, including **sandbox**. You also see **prod_catalog**, because both workspaces share the regional metastore. Access is still controlled by grants. In production, you would also bind **prod_catalog** to the prod workspace only.

12.5 Make an environment-specific change

Add a schema to **environments/dev.tfvars** only:

```
audit = "Audit and compliance data"
```

Then compare the two environments:

```
./deploy.sh dev plan
./deploy.sh prod plan
```

Command	Expected plan	Why
dev plan	1 to add	Only dev's tfvars changed
prod plan	No changes	Prod has its own tfvars and state

12.6 Promote a code change from dev to prod

In **modules/unity_catalog/main.tf**, add a **properties** block to the **databricks_catalog** resource:

```
properties = {
  owner_team = "data-platform"
}
```

Roll it out the way a platform team would:

```
./deploy.sh dev apply     # 1. apply and test in dev (plus the audit schema from 12.5)
./deploy.sh prod plan     # 2. review: the same catalog change appears for prod
./deploy.sh prod apply    # 3. promote to prod
```

Because both environments use the same module, the change you tested in dev is exactly the change that reaches prod.

Step 13: Clean Up Resources

Destroy prod first, then dev, then the state storage:

```
./deploy.sh prod destroy
./deploy.sh dev destroy
az group delete --name rg-tfstate-lab --yes --no-wait
```

Troubleshooting

- **403 error during terraform init** → your **Storage Blob Data Contributor** role on the state account is not active yet, or is missing. Wait a minute and retry.
- **Could not find provider hashicorp/databricks** → **modules/unity_catalog/versions.tf** is missing or has a typo in the **source**.
- **Databricks provider error on the first plan** → you skipped the **-target=module.workspace** bootstrap step for that environment.
- **Plan wants to replace or rename many resources** → the wrong tfvars file is being used with the current state, usually because Terraform was run directly instead of through **deploy.sh**. Do **not** apply. Rerun through the script.
- **State blob is already locked** → another run is in progress, or a previous run crashed. Once you are sure nobody else is running, use **terraform force-unlock** with the lock ID from the error.
- **A catalog, credential or external location already exists** → another user in the same metastore uses the same **project** value. Change **project** in both tfvars files.

Outcomes

You've turned a single-environment lab into a **modular, multi-environment** Terraform project. You now know how to:

- Store state in **Azure Storage** with **Entra ID authentication**, **blob versioning** and automatic **locking**.
- Use **partial backend configuration** to give each environment its own isolated state file.
- Build **network**, **workspace** and **unity_catalog** modules with validated inputs and clear outputs.
- Declare **required_providers** in modules that use non-HashiCorp providers, and pin versions once in the root.
- Apply one **naming** and **tagging** convention across every resource.
- Deploy **dev** and **prod** from identical code, differing only in their **tfvars** files.

- Bootstrap a new environment safely, and promote a tested change from dev to prod.

This sets the foundation for versioning modules with Git tags (for example ?ref=v1.0.0), binding catalogs to specific workspaces, and running plan and apply from a CI/CD pipeline instead of a laptop.